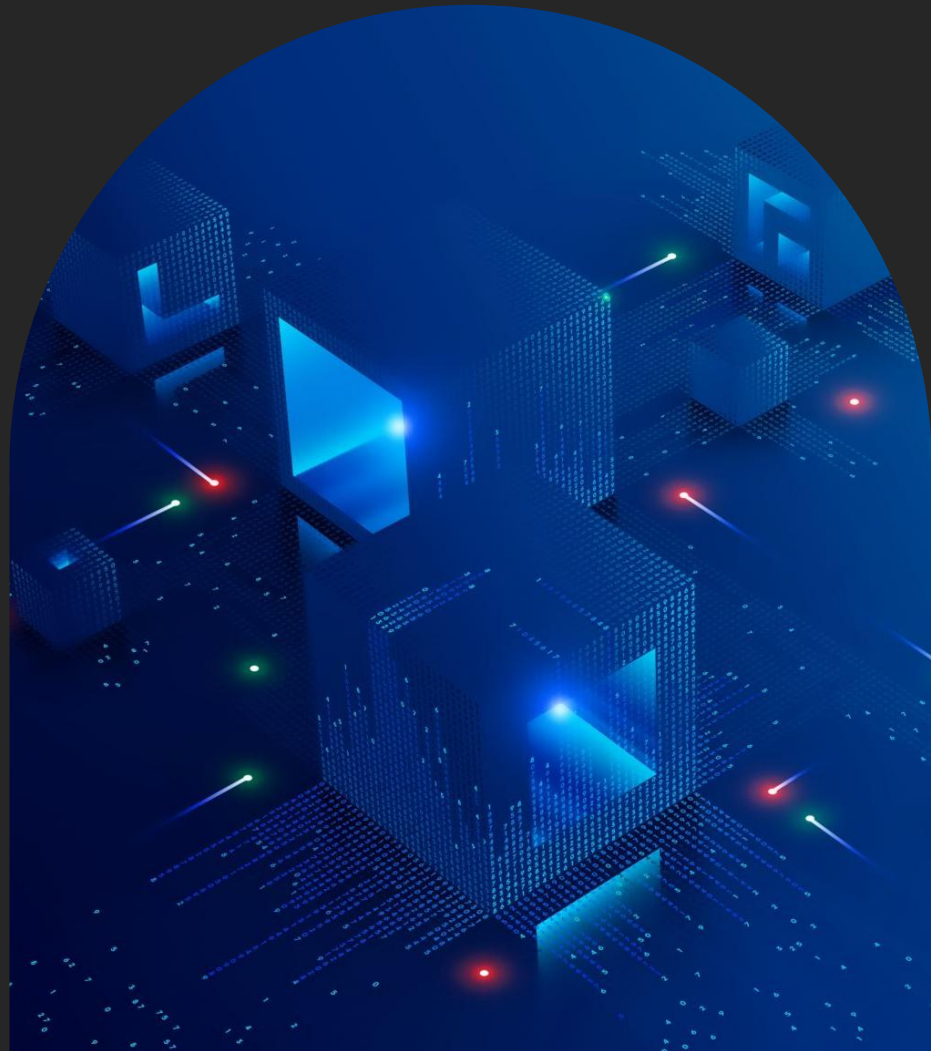




北九州情報IT クリエイター専門学校

秋月蓮音

自己紹介

名前：秋月蓮音

出身：北九州情報ITクリエイター専門学校

趣味・特技：生き物飼育(主に魚)、ゲーム(ELDEN RING, ARK, ポケモン)
龍神流合気道 準師範(歴14年)、キックボクシング(歴7年)

使用言語・ツール

C	:1年
C#	:2年
C++	:2年
Unity	:2年
Git	:1年
TortoiseGit	:1年
Bitbucket	:1年



就活作品

就活作品

作品名：祓魔師(エクソシスト)所・天

ジャンル：3Dアクションゲーム

開発環境：Dxlib,C++

対応機種：PC

制作期間：約4ヶ月

制作時期：2025年10月~2026年2月



作品説明

敵を倒して道具を集めよう



ギミックを解いてお宝集めよう



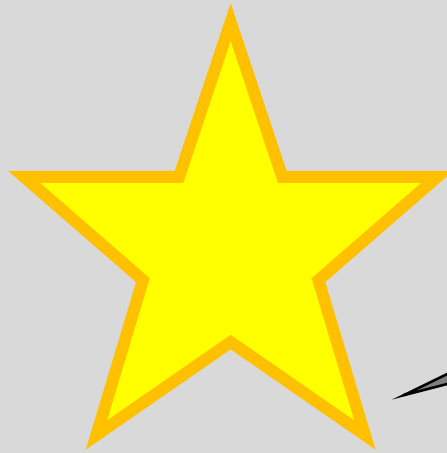
お宝を納品してゴールを探そう



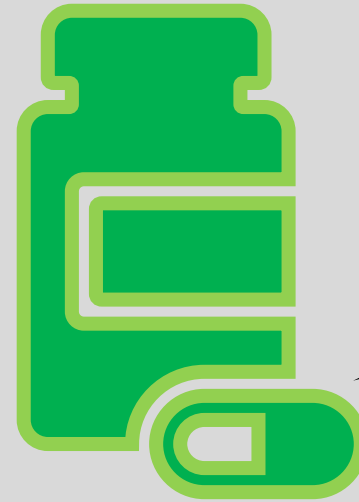
技術紹介

技術紹介 1: アイテム

アイテムの見た目と機能を別々に作ることによってカスタマイズできるようになった例)



見た目 : お宝
中身 : 薬



見た目 : 薬
中身 : お宝

技術紹介 1: アイテム

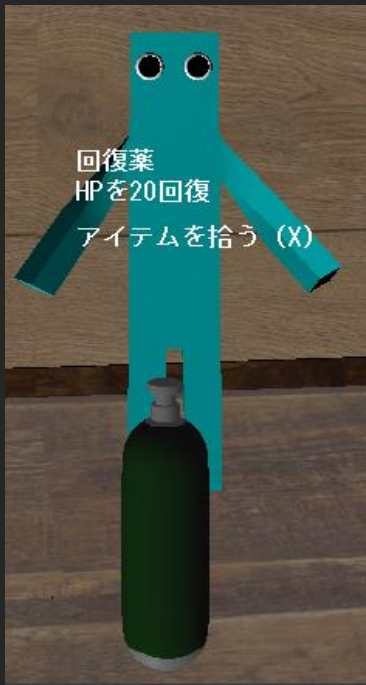
アイテムの見た目と機能を別々に作ることによってカスタマイズできるようになった

```
SpawnItem(std::make_unique<MedicineItem>①
```

```
② ("回復薬", "HPを10回復", ItemKey::MEDICINE_HEAL, std::make_unique<Heal>(10)),
```

```
③ VGet(46.0f, 1.0f, 75.0f), ItemKey::MEDICINE_HEAL, true, respawnTime);
```

- ①・・・アイテム機能のカテゴリから絞り込めるようになっている
- ②・・・アイテムの表示名、説明、インベントリでの画像、機能と効果量
- ③・・・出現位置、フィールド上の見た目、アイテムの復活フラグ、復活タイマーとなっていて細かく設定でき拡張もしやすくなっている



アイテムの
見た目は同じ
効果が異なる



技術紹介 2: 外部ファイル化

敵の初期ステータスやギミックの初期設定などをjsonファイルに書き
ゲーム内で読み込ませる

```
{
  "enemies": [
    {
      "id": "LadderEnemy",
      "name": "はしこ",
      "hp": 5,
      "speed": 0.2,
      "melee": 4,
      "SphereCollisionPosition": { "x": 0.0, "y": 0.0, "z": 0.0 },
      "SphereCollisionRadius": 1,
      "detectionRange": 25.0,
      "attackInterval": 40,
      "attackRange": 6,
      "attackHitStart": 50,
      "attackHitEnd": 100,
      "attackSphereCount": 4,
      "attackSphereSpacing": 1.5,
      "attackOffsetX": 0.0,
      "attackOffsetY": 2.0,
      "attackOffsetZ": 1.0,
      "attackSphereRadius": 1.0,
      "dropItem": {
        "type": "Tool",
        "name": "はしこ",
        "description": "はしこを修理できる",
        "value": 0,
        "effect": "Ladder"
      }
    }
  ],
}
```

▲jsonファイルにデータを入力

```
for (auto& enemyData : data["enemies"])
{
    EnemyParams params;
    params.id = Utf8ToSjis(enemyData["id"]);
    params.name = Utf8ToSjis(enemyData["name"]);
    params.hp = enemyData["hp"];
    params.speed = enemyData["speed"];
    params.melee = enemyData["melee"];
    auto& pos = enemyData["SphereCollisionPosition"];
    float sx = pos.value("x", 0.0f);
    float sy = pos.value("y", 0.0f);
    float sz = pos.value("z", 0.0f);
    params.SphereCollisionPosition = VGet(sx, sy, sz);
    params.SphereCollisionRadius = enemyData["SphereCollisionRadius"];
    params.detectionRange = enemyData["detectionRange"];
    params.attackInterval = enemyData["attackInterval"];
    params.attackRange = enemyData["attackRange"];
    params.attackHitStart = enemyData["attackHitStart"];
    params.attackHitEnd = enemyData["attackHitEnd"];
    params.attackOffsetX = enemyData["attackOffsetX"];
    params.attackOffsetY = enemyData["attackOffsetY"];
    params.attackOffsetZ = enemyData["attackOffsetZ"];
    params.attackSphereCount = enemyData["attackSphereCount"];
    params.attackSphereSpacing = enemyData["attackSphereSpacing"];
    params.attackSphereRadius = enemyData["attackSphereRadius"];
```

▲jsonファイルからデータを読み込む

技術紹介 3: ギミック

道具を使用することで解除できるギミックはプレイヤーのインベントリを参照して解除できるか判定する

```
// 「Xボタンで使用」などのインタラクトUI
// Xボタンを押したら発動するようにする
if (player->IsInteractPressed() && dist < INTERACT_RANGE)
{
    OnPlayerInteract(player);
    player->GetInventory()->RemoveToolItem(); // 道具を消費
}
```



プレイヤーの
インベントリを参照する

```
void HammerGimmick::OnPlayerInteract(Player* player)
{
    // 効果演出
    EffectManager::GetInstance()->PlayEffect(EFFECT_HAMMER_USED, m_Pos);
    SoundManager::GetInstance()->PlaySE(SE_TYPE_GIMMICK_HAMMER);
    // 一度使ったら非表示
    m_Active = false;
    m_IsUsed = true;
}
```

```
void DoorGimmick::OnPlayerInteract(Player* player)
{
    m_IsUsed = true;
    // 効果演出
    SoundManager::GetInstance()->PlaySE(SE_TYPE_GIMMICK_DOOR_OPEN);
    // 一度使ったら非表示（または消す）
    m_Active = false;
}
```



各ギミックは
GimmickBaseを継承して
いるので拡張する際も簡
単にできるので効率がい
い！

技術紹介 4:ギミック配置

ギミックの配置などはjsonファイルで管理しているのでプログラムを触らずとも簡単に新しく生成などが行える

```
// JSONからギミック配置
const auto& gimmickParams = GimmickParamsLoader::GetAll();

for (const auto& param : gimmickParams)
{
    GimmickBase* g = CreateGimmick(param.type);
    g->Load(); // モデル読み込み
    g->SetParams(param); // パラメータを設定
    g->SetPos(param.pos); // 配置位置を設定
    g->SetRequiredTool(param.tool); // ギミックごとにツール条件を設定できるようにする
    g->SetRequiredTool2(param.tool2); // ギミックごとにツール条件2を設定できるようにする
    g->Init(); // コリジョンなどの初期化
}
```

(jsonファイル)
▼このような行で生成ができる

```
{
    "type": "BAR_SAW_GIMMICK",
    "tool": "BAR",
    "tool2": "SAW",
    "position": [-50.0, -0.5, 50.0],
    "aabbHalfX": 5.0,
    "aabbHalfY": 9.0,
    "aabbHalfZ": 0.5,
    "aabbOffsetX": 0.0,
    "aabbOffsetY": 3.0,
    "aabbOffsetZ": 0.0
},
```

▲ギミックの生成プログラム

技術紹介 5:描画距離

各オブジェクトはプレイヤーからの距離によって描画をするか決める事により重い処理であるDrawを軽くすることができる

```
void StageObjectManager::Draw()
{
    Player* player = PlayerManager::GetInstance()->GetPlayer();
    VECTOR playerPos = player->GetPos();

    for (auto obj : m_StageObjects)
    {
        VECTOR pos = obj->GetPos();

        float dist = VSize(VSub(playerPos, pos));

        if (dist > STAGEOBJECT_DISTANCE) continue; // 遠いから描画しない
        obj->Draw();
    }
}
```



プレイヤーの座標取得



オブジェクト座標を取得しプレイヤーとの距離を計算する



計算された距離が指定距離内なら描画する

▲距離によって描画を変える処理(ステージオブジェクト)

この処理は複数の描画をしないといけないDrawで使っている

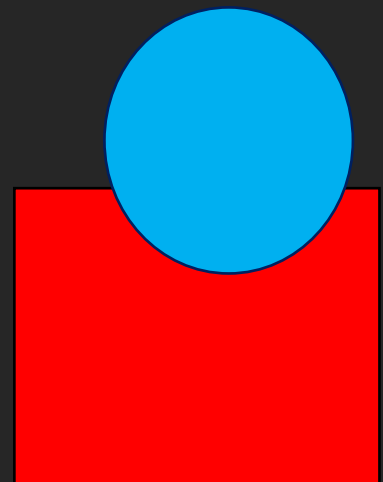
技術紹介 6: 当たり判定

AABB対**AABB**や**球**対**球**、**AABB**対**球**
などの様々な当たり判定を実装した

```
// AABBとSphere 判定
bool CollisionManager::CheckAABBSphereCollision(CollisionAABB* aabb, CollisionSphere* sphere)
{
    VECTOR aabbCenter = VAdd(aabb->GetTargetPos(), aabb->GetLocalPos()); // AABBの中心座標
    VECTOR aabbHalfSize = VScale(aabb->GetSize(), 0.5f); // AABBの半分のサイズ

    VECTOR sphereCenter = VAdd(sphere->GetTargetPos(), sphere->GetLocalPos()); // 球の中心座標
    float radius = sphere->GetRadius(); // 球の半径

    // AABBの各軸における最近接点を求める
    float dx = fmaxf(aabbCenter.x - aabbHalfSize.x, fminf(sphereCenter.x, aabbCenter.x + aabbHalfSize.x)) - sphereCenter.x;
    float dy = fmaxf(aabbCenter.y - aabbHalfSize.y, fminf(sphereCenter.y, aabbCenter.y + aabbHalfSize.y)) - sphereCenter.y;
    float dz = fmaxf(aabbCenter.z - aabbHalfSize.z, fminf(sphereCenter.z, aabbCenter.z + aabbHalfSize.z)) - sphereCenter.z;
    // 最近接点と球の中心との距離の二乗を計算
    float distSq = dx * dx + dy * dy + dz * dz;
    // 距離の二乗が半径の二乗以下なら当たっている
    return distSq <= radius * radius;
}
```



▲AABBと球の当たり判定プログラム

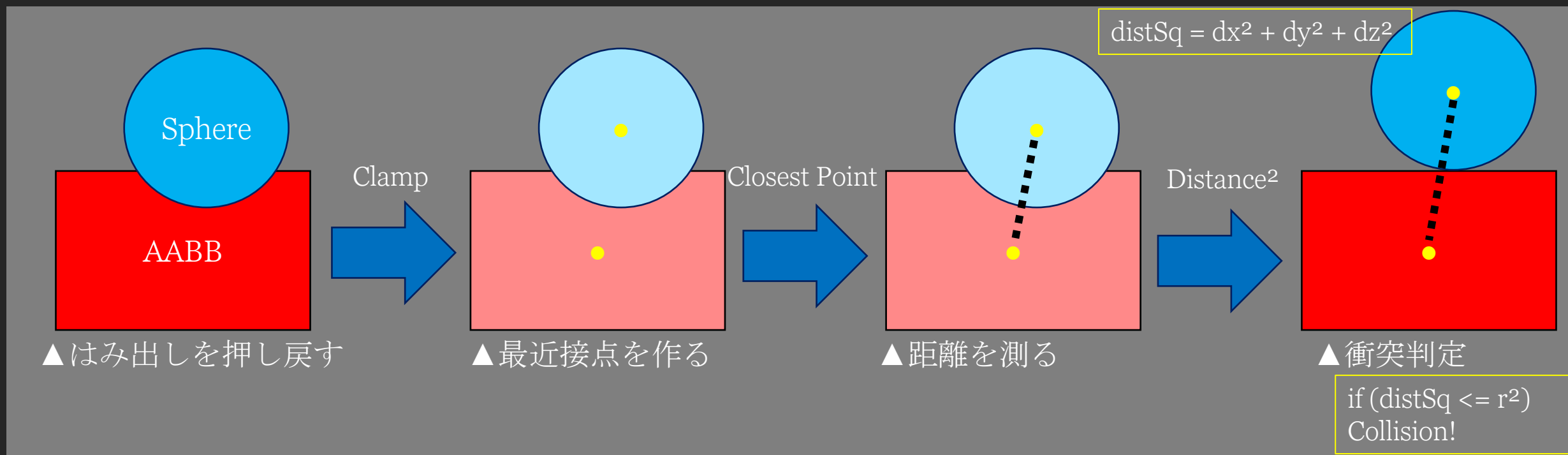
プレイヤー : AABB

敵や敵の攻撃 : 球 などの判定に使っている

技術紹介 6: 当たり判定

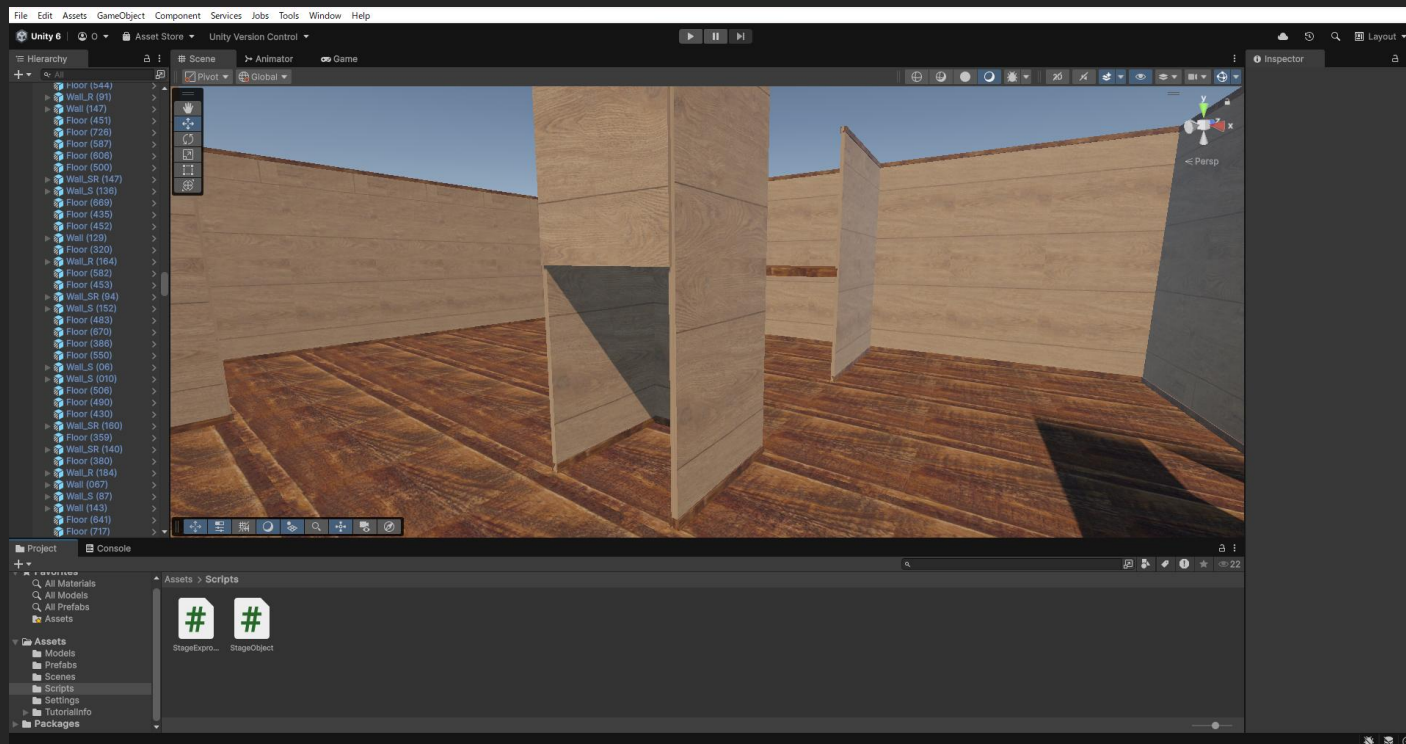
図で計算の説明

▼ **AABB**と**球**が衝突した場合

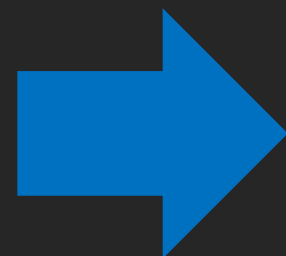


技術紹介 7: ステージ制作

Unityをステージエディターとして使用



▲Unity でステージを制作



オブジェクトのID
ポジション
ローテーション



jsonファイルに書き込む

技術紹介 7: ステージ制作

Unityをステージエディターとして使用

```
for (const GameObject& obj : m_Objects)
{
    // 天井
    if (obj.id >= static_cast<int>(LocateObjectID::CEILING_00))
    {
        int id = obj.id - static_cast<int>(LocateObjectID::CEILING_00);
        StageObjectManager::GetInstance()->CreateCeiling(id, obj.pos, obj.rot, obj.scale);
    }
    // ドア
    else if (obj.id >= static_cast<int>(LocateObjectID::DOOR_00))
    {
        int id = obj.id - static_cast<int>(LocateObjectID::DOOR_00);
        StageObjectManager::GetInstance()->CreateDoor(id, obj.pos, obj.rot, obj.scale);
    }
    // 壁
    else if (obj.id >= static_cast<int>(LocateObjectID::WALL_00))
    {
        int id = obj.id - static_cast<int>(LocateObjectID::WALL_00);
        StageObjectManager::GetInstance()->CreateWall(id, obj.pos, obj.rot, obj.scale);
    }
    // 床
    else if (obj.id >= static_cast<int>(LocateObjectID::FLOOR_00))
    {
        int id = obj.id - static_cast<int>(LocateObjectID::FLOOR_00);
        StageObjectManager::GetInstance()->CreateFloor(id, obj.pos, obj.rot, obj.scale);
    }
}
```

▲IDに対応したオブジェクトを生成する

▼ステージ完成



制作実績

制作実績① 個人制作

作品名：夜を啄む

ジャンル：ホラー風探索ゲーム

制作環境：Unity,C#

制作期間：約2週間

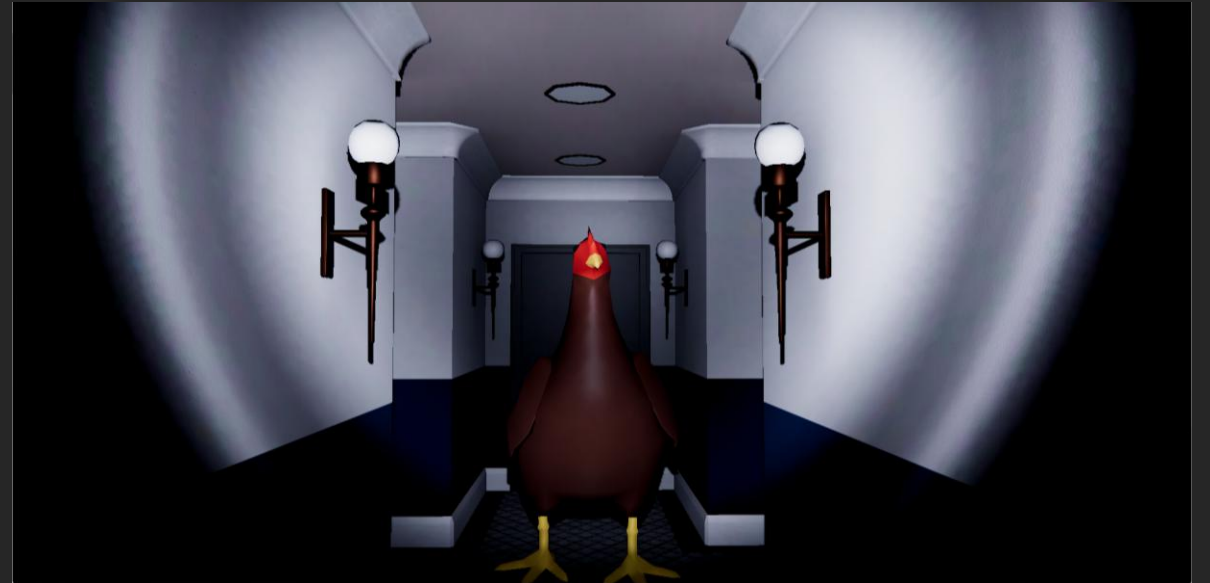
制作時期：2026年2月

目的、学んだ事：

- ・HDRP の機能について
- ・Fog
- ・レイトレーシング
- ・タイムラインの使い方

建物の中に入ろう

何の音だ？
見に行ってみよう



制作実績② 個人制作

作品名：とある夜の田舎にて

ジャンル：HD2D風探索ゲーム

制作環境：Unity,C#

制作期間：約2週間

制作時期：2025年12月末
～2026年1月中旬

目的、学んだ事：

- ・ポストプロセスを使った演出
- ・csvファイルを使った外部データ化



制作実績③ チーム制作

作品名：ギミックラッシュ！

ジャンル：3Dアクションレースゲーム

制作環境：Unity,C#

制作期間：2ヶ月

制作時期：2025年6月

～2025年8月

担当：

- ・タイトル、クレジット、ステージ1
- ・カメラ操作、シーン遷移
- ・ローカルマルチプレイ機能

目的、学んだ事：

- ・チーム内での報連相の大事さ
- ・TortoiseGit ,Bitbucketの使い方



制作実績④ 個人制作

作品名 : Hit&Blow!

ジャンル : 推理パズルゲーム

制作環境 : DxLib,C++

制作期間 : 約1ヶ月

制作時期 : 2026年9月

目的、学んだ事 :

- ・ネットワーク通信の仕組み
- ・クライアントとサーバーの役割を分ける設計
- ・可変引数と文字列処理



その他

今後の目標

1. チームとして貢献できるプログラマーを目指す

以前のチーム制作の際に、自分のプログラムが分かりにくかったのでチーム制作の連携効率が上昇すると思った。

2. コミュニケーション能力の向上

以前のチーム制作でGitで衝突が大量に発生してしまった。
制作を円滑に進めるため、この力の向上を計る。

3年生での卒業研究でUnreal Engineでのチーム制作を計画しておりこれらに対しての改善を目指す。

3年時の学習計画

1.Unreal Engineの使い方を覚える

2.ネットワークプログラミング

を頑張ってオンラインプレイが
できるゲームを制作したい！